

LoRa Communication with Two ESP32

Renato Meneghin

PhD. Candidate Student

UFSC - Federal University of Santa Catarina

Florianópolis, Brasil

renatoschenkel16@gmail.com

Abstract—This report presents the development of an embedded sensing and telemetry system based on two ESP32 microcontrollers and LoRa communication. The transmitter node integrates a BME280 environmental sensor and an MPU6050 inertial sensor, with data acquisition tasks implemented using FreeRTOS. Accelerometer data are processed locally through a FIR low-pass filter and a TinyML motion classifier, reducing the amount of information transmitted through the wireless link. A binary telemetry frame with CRC protection is transmitted through the SX1276 LoRa transceiver to a second ESP32 acting as receiver. Experimental tests demonstrated the complete acquisition, processing, classification, transmission and reception chain, including the validation of the telemetry frame and its integrity through CRC verification.

Index Terms—Embedded Systems, Internet of Things, LoRa, FreeRTOS, TinyML, Digital Signal Processing.

““latex

I. INTRODUCTION

Embedded systems are an important component of modern Internet of Things (IoT) applications. This is driven by increasing demands of sensor integration, processing and communication capabilities. In these systems, the sensor nodes acquire the physical measurements, process them locally, and transmit the resulting data through communication networks. These systems, however, are constrained by computational resources, memory, energy consumption, and communication bandwidth [1].

In wireless sensing applications, transmitting all the data acquired by a sensor may result in unnecessary communication overhead. Local processing can extract relevant information before transmission, reducing the data length exchanged between nodes. This approach is particularly relevant for wireless technologies designed with low-data-rate and long-range communication. Therefore, integrating LoRa communication with edge computing has been investigated for IoT systems, where sensing and data processing can be performed close to the source [2].

Modern microcontrollers can also execute machine learning algorithms. This is enabled by their increased processing capabilities and the use of the Tiny Machine Learning (TinyML) approach, which targets machine learning inference on resource-constrained devices. This approach is particularly suitable for motion and image recognition applications, where relevant information can be represented by compact classification results [3].

Based on these concepts, this work presents the implementation of an embedded sensing and telemetry system using two ESP32-based LoRa nodes. The transmitter node integrates a BME280 environmental sensor and an MPU6050 inertial sensor. The firmware is organized using the FreeRTOS real-time operating system, separating sensor acquisition, digital signal processing, machine learning inference, and wireless transmission into independent tasks. The inertial data is processed locally using a finite impulse response (FIR) filter and a TinyML classifier, while the environmental measurements are directly incorporated into the telemetry data. The resulting information is organized into a compact binary frame protected by a cyclic redundancy check (CRC) and transmitted through an SX1276 LoRa transceiver.

The main objective of the project is to demonstrate the integration of sensing, real-time task management, digital signal processing, machine learning, and wireless communication in a resource-constrained embedded system. The following sections describe the hardware components, firmware architecture, signal processing and machine learning implementation, experimental setup, and results obtained from the communication between the two nodes. ““

II. COMPONENTS AND HARDWARE

The transmitter node consists of a Heltec WiFi LoRa 32 (V2) [4] development board, the BME280 sensor module [5], and the MPU6050 accelerometer sensor module [6]. Both sensors are connected using the I²C bus. A UML diagram of the transmitter node is shown in Figure 1.

In contrast, the receiver node consists only of the Heltec development board connected to a computer. A UML diagram of the transmitter node is shown in Figure 1.

The Heltec WiFi Lora 32 development board unifies an ESP32 microcontroller and an SX1276 LoRa transceiver. This combination allows for faster development, prototyping, and implementation of LoRa communication. The BME280 acts as a microclimate monitor, while the MPU6050 measures board movement and being used for a possible fusion with a GNSS sensor for positioning tracking.

III. FIRMWARE

A. Transmitter

The transmitter was separated into 4 tasks. Two are responsible for sensor data acquisition, one for the DSP, and the last to transmit the packet. The RTOS architecture for the

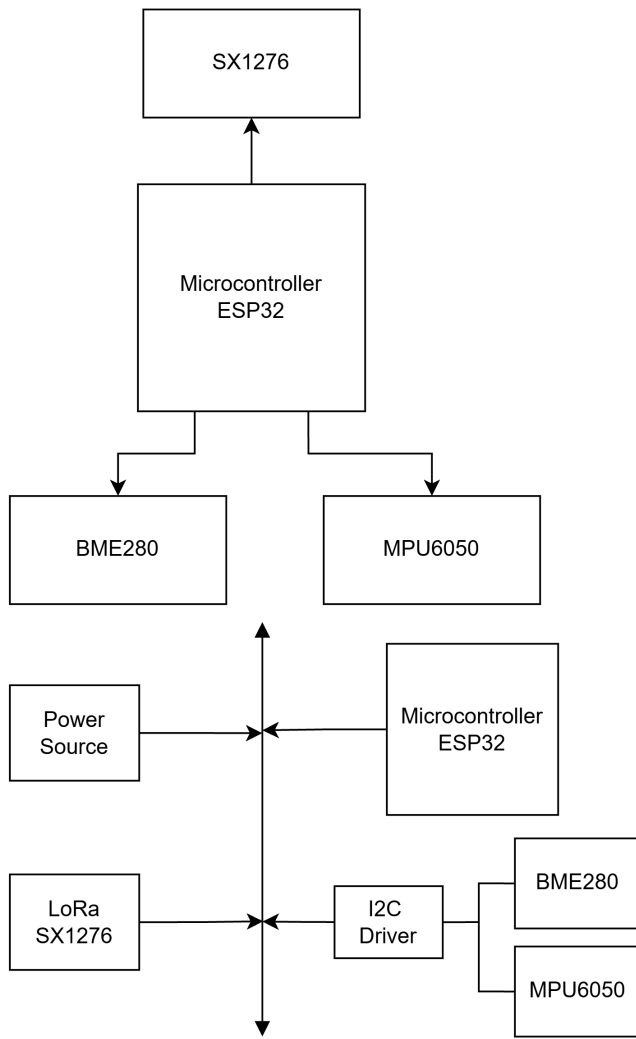


Fig. 1. UML describing the Transmitter components

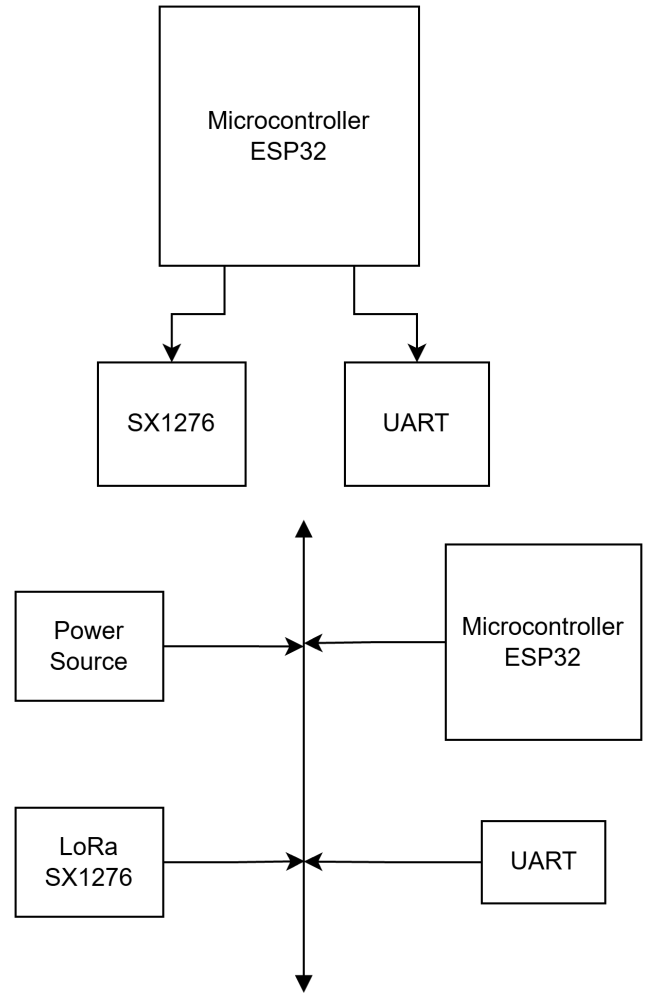


Fig. 2. UML describing the Receiver components

transmitter node is illustrated in Figure 3. A data structure is used to store the values in memory after the acquisition and DSP. The transmission task retrieves the data and mounts the transmission frame. The tasks for meteorological monitoring, accelerometer, DSP and LoRa transmitter are demonstrated in Figures 4, 5, 6 and 7, respectively.

The Meteorological and Acceleration acquisition and transmission tasks are triggered by the FreeRTOS timer, while the DSP task is called by the Acceleration queue. The sample rates are 1 Hz for the meteorological data and 1 kHz for the inertial data. The meteorological data is stored in the packet structure, while the acceleration data is sent to the DSP task, through a FreeRTOS Queue. Figure 1 shows the Sequence Diagram of the Transmitter.

Public libraries were used for each module. The exceptions are: The Acceleration task initialization required waking up the MPU6050 directly from the I²C driver. The FIR filter implemented with a cutoff frequency of 150 Hz and a bandwidth

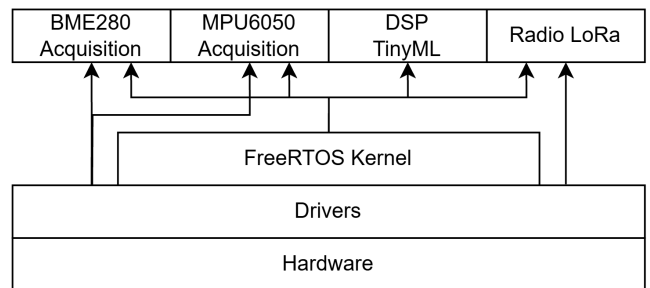


Fig. 3. Transmitter RTOS Architecture

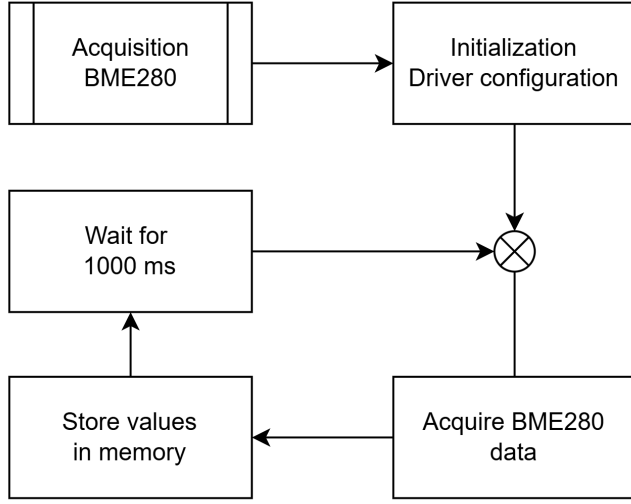


Fig. 4. BME280 data acquisition task on the Transmitter node

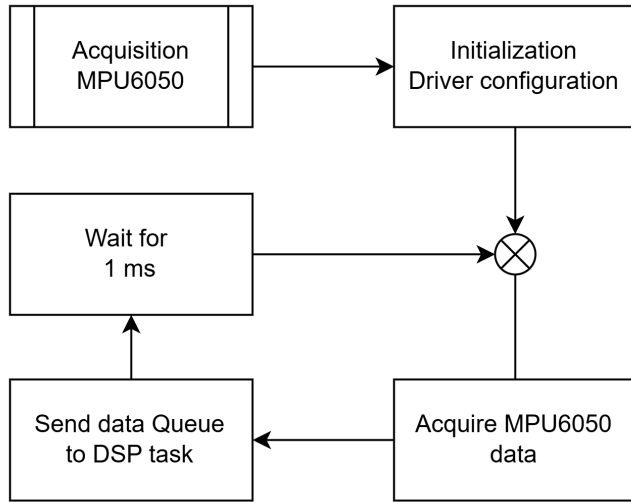


Fig. 5. MPU6050 acquisition task on the Transmitter node

of 100 Hz. Using the FIIR Tool [7], a FIR filter was designed with 31 coefficients. The frequency response of the FIR filter is illustrated in Figure 9. The absolute values of the filtered and unfiltered accelerations are stored in the data structure.

The TinyML was implemented in the DSP task. It was designed with the help of the EdgeImpulse tutorial on motion recognition [8]. The parameters were modified to support the accelerometer sample rate. The project was then exported as an Arduino library and imported into the project. When the firmware runs the classifier, it detects whether the object is idle, in an incident or running. The resulting label is sent to the packet data structure.

The transmitter task sends the data to the LoRa transmitter. It retrieves the data stored in the data packet structure, mounts the transmission frame, calculates the CRC and transmits to the SX1276 LoRa transceiver.

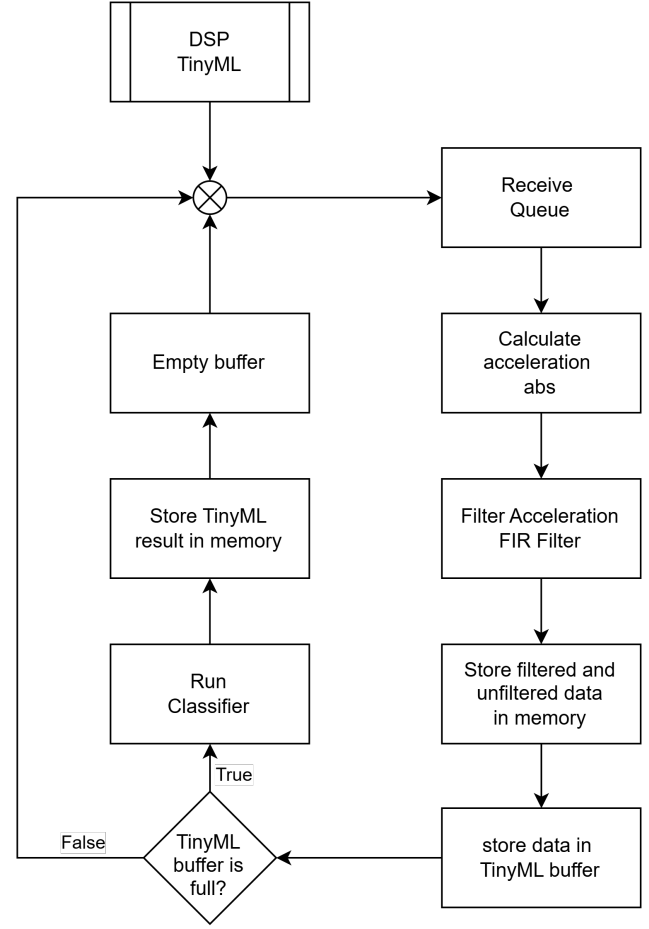


Fig. 6. DSP and the TinyML Task on the Transmitter node

B. Receiver

The receiver used a simple for loop with a data processing function. The processing function extracts the data from the received frame, checks the CRC, and transmits it to the host computer. Figure 10 illustrates the firmware used in the Receiver microcontroller.

IV. EXPERIMENTAL RESULTS

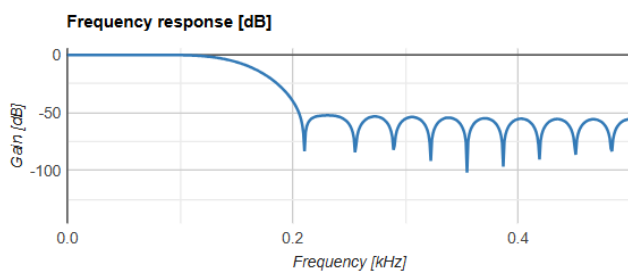
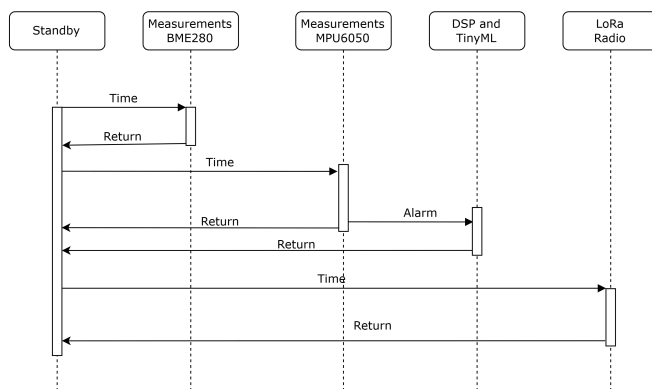
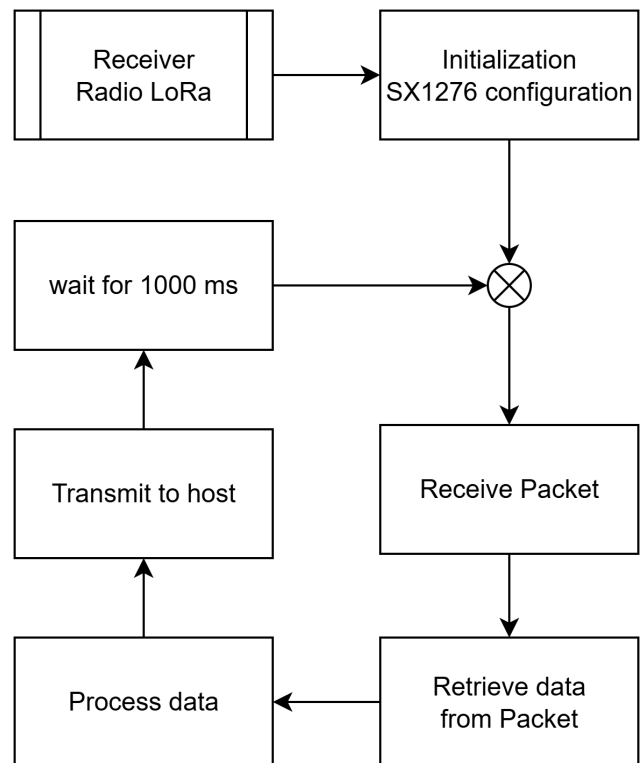
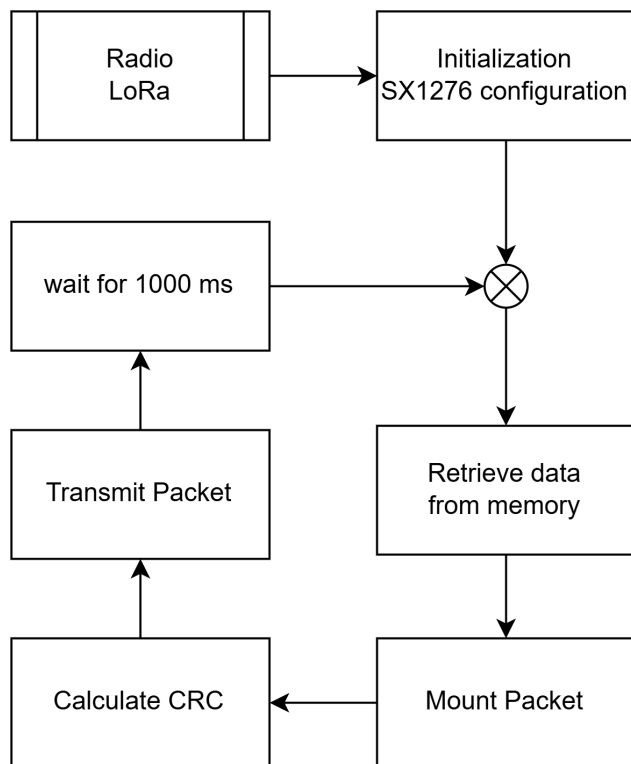
A. Setup

A setup with two breadboards was used to test the LoRa communication. Figure 11 depicts the configuration implemented.

B. Transmitter

For the transmitter node, the serial port is used to check the parameters sent. It sends the meteorological and acceleration data, the classifier result label, the calculated CRC and if the LoRa transmission was successful. A log is used to debug the transmitter. Figure 12 illustrates the output log.

The output log indicates some of the tasks outputs. The meteorological and acceleration data from the transmitter task are in integer format times one hundred, as compared



```
Transmitting LoRa packet...
TX: T=1968 C, P=102574 hPa, A=9742 g, Af=10690 g
Received CRC: 0x8A3E | Calculated CRC: 0x8A3E
Received CRC size: 2 | Received frame size: 22
TelemetryFrame size: 22
CRC offset: 20
LoRa TX successful (22 bytes)
Predictions: Idle detected
BMP280: Temp=19.68 C, Pressure=1025.72 hPa, Altitude=-103.31 m
Transmitting LoRa packet...
TX: T=1968 C, P=102572 hPa, A=9729 g, Af=10689 g
Received CRC: 0x1DF1 | Calculated CRC: 0x1DF1
Received CRC size: 2 | Received frame size: 22
TelemetryFrame size: 22
CRC offset: 20
LoRa TX successful (22 bytes)
BMP280: Temp=19.69 C, Pressure=1025.73 hPa, Altitude=-103.38 m
Transmitting LoRa packet...
TX: T=1969 C, P=102572 hPa, A=9769 g, Af=10685 g
Received CRC: 0xDADB | Calculated CRC: 0xDADB
Received CRC size: 2 | Received frame size: 22
TelemetryFrame size: 22
CRC offset: 20
LoRa TX successful (22 bytes)
```

```

Transmitting LoRa packet...
TX: T=1968 C, P=102574 hPa, A=9742 g, Af=10690 g
Received CRC: 0x8A3E | Calculated CRC: 0x8A3E
Received CRC size: 2 | Received frame size: 22
TelemetryFrame size: 22
CRC offset: 20
LoRa TX successful (22 bytes)
Predictions: Idle detected
BMP280: Temp=19.68 C, Pressure=1025.72 hPa, Altitude=-103.31 m
Transmitting LoRa packet...
TX: T=1968 C, P=102572 hPa, A=9729 g, Af=10689 g
Received CRC: 0x1DF1 | Calculated CRC: 0x1DF1
Received CRC size: 2 | Received frame size: 22
TelemetryFrame size: 22
CRC offset: 20
LoRa TX successful (22 bytes)
BMP280: Temp=19.69 C, Pressure=1025.73 hPa, Altitude=-103.38 m
Transmitting LoRa packet...
TX: T=1969 C, P=102572 hPa, A=9769 g, Af=10685 g
Received CRC: 0xDADB | Calculated CRC: 0xDADB
Received CRC size: 2 | Received frame size: 22
TelemetryFrame size: 22
CRC offset: 20
LoRa TX successful (22 bytes)

```

Fig. 12. transmitter node Log frames

```

[SX1276] Waiting for incoming packet ... success!
[SX1276] Data: Received CRC: 0x8E0B | Calculated CRC: 0x8E0B
Received CRC size: 2 | Received frame size: 22

===== TELEMETRY =====
Timestamp : 49935 ms
Temperature: 22.86 C
Pressure : 1025.39 hPa
Altitude : 554.71 m
Accel : 97.30 g
Accel FIR : 97.22 g
Event : NORMAL (0x01)
CRC : 0x8E0B [OK]

=====
[SX1276] RSSI: -99.00 dBm
[SX1276] SNR: -1.00 dB
[SX1276] Waiting for incoming packet ... timeout!
[SX1276] Waiting for incoming packet ... timeout!
[SX1276] Waiting for incoming packet ... timeout!
[SX1276] Waiting for incoming packet ... success!
[SX1276] Data: Received CRC: 0xFAD1 | Calculated CRC: 0xFAD1
Received CRC size: 2 | Received frame size: 22

===== TELEMETRY =====
Timestamp : 51031 ms
Temperature: 22.85 C
Pressure : 1025.40 hPa
Altitude : 554.64 m
Accel : 97.66 g
Accel FIR : 97.47 g
Event : NORMAL (0x01)
CRC : 0xFAD1 [OK]

=====
[SX1276] RSSI: -99.00 dBm
[SX1276] SNR: 2.25 dB

```

Fig. 13. Receiver node log frame

with the meteorological task data log. The conversion was used to reduce the number of bytes used for the telemetry and to avoid floating point transmission. The classifier log demonstrates that it identifies the object in "idle" state. From the transmission task, the microcontroller sends 22 bytes to the LoRa transceiver, being 20 bytes of data and 2 bytes of CRC.

C. Receiver

The receiver node transmits the received frame to the host computer via serial communication. Figure 13 illustrates the receiver log sent to the host.

After receiving the data, the receiver node reconstructed the meteorological and acceleration data into floating point formats, with the timestamp and the classifier result. The log also identifies the CRC, RSSI and SNR of the signal.

V. DISCUSSIONS

The use of the Heltec Wifi LoRa boards grouped together many elements required for the IoT project. With integrated antennas, transceiver chip and a microcontroller, only the hardware sensor modules and the firmware were acquired and connected on the breadboard. This makes prototype design faster and easier to deploy before reaching the final product.

The FreeRTOS architecture was used to study its functions and implementations. Embedded Systems use RTOS to guarantee that critical tasks complete within strict and predictable time limits [9]. The separation of firmware in tasks reduces code complexity. The semaphore allows safe data storage and sharing between multiple tasks. The queue can transmit data between tasks within a specific time frame.

Both sensors use the I²C communication protocol and have different device addresses, allowing them to share the same bus and the programmer to configure one driver instead of two. Both were successfully implemented with libraries, although the accelerometer required waking up via an external driver command.

The DSP was originally planned to be implemented via the esp-dsp library. However, since some libraries required the C++ framework, there compilation incompatibilities. To bypass this limitation, the FIR filter was directly implemented in the firmware. This, in turn, allowed the study and use of a ring buffer for the filter.

The use of the DSP and machine learning classifier reduced the amount of inertial data required for telemetry from 24 bytes to 5 bytes. If the accelerometer full data was transmitted, it would be necessary to transmit 24 bytes (6 floating points of 4 bytes each). The compression of the analysis directly on the transmitter node and subsequent data conversion reduced it to five bytes, consisting of two for the unfiltered and filtered acceleration data, each, and one for the classifier label. This reduces the transmission data length, time and energy consumed in the telemetry packet.

VI. CONCLUSION

This report presents an implementation of communication between two microcontrollers with LoRa. It used two LoRa development boards, one implemented as a sensor node and another as the receiver node.

This project also implemented wired communication protocols, such as I²C for the sensors, SPI for the LoRa transceiver and UART for the host computer. A FreeRTOS architecture was used to separate each task and reduce code complexity, and allow for safe data managing between tasks. A machine learning classifier was also implemented, identifying the movement state of the transmitter node.

Future works includes the development of a host application, integration with LoRaWAN and IoT devices, and expansion of the sensor node with GNSS and additional sensors.

REFERENCES

- [1] Lorik Fetahu, Arianit Maraj, and Abdullah Havolli. “Internet of Things (IoT) benefits, future perspective, and implementation challenges”. In: *2022 45th Jubilee International Convention on Information, Communication and Electronic Technology (MIPRO)*. 2022, pp. 399–404. DOI: 10.23919/MIPRO55190.2022.9803487.
- [2] Ben Foubert and Bart Nauwelaers. “A Survey on LoRa for IoT: Integrating Edge Computing”. In: *2019 Fourth International Conference on Fog and Mobile Edge Computing (FMEC)*. 2019, pp. 1–6. DOI: 10.1109/FMEC.2019.8795313.
- [3] ... “Internet of Things Challenges and the Emerging Technology of TinyML”. In: *2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. 2023, p. ... DOI: 10.1109/DCOSS-IoT58021.2023.00082.
- [4] Heltec. *WiFi LoRa 32(V2), ESP32 + SX127X LoRa Node, Meshtastic and LoRaWAN compatible — Phaseout — heltec.org*. <https://heltec.org/project/wifi-lora-32v2/>. [Accessed 07-09-2026]. 2026.
- [5] Bosch Sensortec. *BME280: Combined humidity, pressure and temperature sensor*. Document number: BST-BME280-DS002. Bosch Sensortec GmbH. URL: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>.
- [6] SunFounder. *Accelerometer & Gyroscope Module (MPU6050)*. Accessed: 2026-09-06. SunFounder. 2025. URL: https://docs.sunfounder.com/projects/ultimate-sensor-kit/en/latest/components_basic/05-component_mpu6050.html.
- [7] Tom Roelandts. *FIIIR! - Design FIR & IIR Filters*. <https://fiiir.com/>. Acessado em: 7 de setembro de 2026. 2026.
- [8] Edge Impulse. *Motion Recognition with Anomaly Detection*. Acessado em: 7 de setembro de 2026. Edge Impulse Documentation. 2026. URL: <https://docs.edgeimpulse.com/tutorials/end-to-end/motion-recognition>.
- [9] Pavel Moroshek. *Why Timing Matters: The Role of RTOS in Advanced Embedded Systems*. Acessado em: 7 de setembro de 2026. Promwad. Nov. 2024. URL: <https://promwad.com/news/rtos>.